NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Bantak Emil*
ID: sdi1800122

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025-2026*

Contents

1 Abstract **2**

2 Data processing and analysis **2**
2.1 Data Cleaning . . . 2
2.2 Text Preprocessing . . . 2
2.3 Analysis . . . 3
2.4 Data partitioning for train, test and validation . . . 10
2.5 Vectorization . . . 11

3 Algorithms and Experiments **12**
3.1 TF-IDF + Logistic Regression Part . . . 12
3.1.1 Table of trials TF-IDF + Logistic Regression Part . . . 14
3.2 Word2Vec + Logistic Regression Part . . . 14
3.2.1 Table of trials Word2Vec + Logistic Regression Part . . . 15
3.2.2 Analysis of Experimental Results . . . 17
3.3 Hyper-parameter tuning . . . 21
3.3.1 TF-IDF + Logistic Regression . . . 21
3.3.2 Word2Vec + Logistic Regression . . . 22
3.4 Optimization techniques . . . 23
3.5 Evaluation . . . 26
3.5.1 Learning Curve . . . 27
3.5.2 TF-IDF + Logistic Regression Part . . . 28
3.5.3 Word2Vec + Logistic Regression Part . . . 29
3.5.4 Confusion matrix . . . 30

4 Results and Overall Analysis **33**
4.1 Results Analysis . . . 33
4.1.1 Best trial . . . 34

5 Bibliography **36**

1. Abstract

This report presents the development and evaluation of classifiers for response clarity classification in political question–answer (QA) pairs. The task uses the dataset released for SemEval 2026 Task 6 (CLARITY) and wants us to predict whether a president's answer to a journalist's question is a Clear Reply, an Ambivalent response, or a Clear Non-Reply.

To tackle this three-class classification problem two text representation approaches are implemented each combined with a Logistic Regression classifier. The first representation uses TF-IDF features. The second representation uses Word2Vec embeddings trained from scratch on the task dataset, where each QA pair is encoded as the average of its token vectors. Both approaches are evaluated and compared using macro F1-score on the test set, which equally weights all three classes regardless of their frequency in the dataset.

Prior to model training the dataset undergoes data cleaning and text preprocessing. For both approaches hyperparameter optimization is performed via GridSearchCV configured with 5-fold cross-validation, while Word2Vec-specific parameters such as vector size, window size and number of training epochs are also optimized.

2. Data processing and analysis

2.1. Data Cleaning

The first cleaning step involves identifying and removing (question, answer) pairs with conflicting clarity labels. Since the dataset was annotated by multiple annotators the same (question, answer) pair could receive different clarity labels from different annotators. Keeping such pairs would introduce noise into the training process, so for the same (question, answer) pair if the annotators did not agree on the same clarity label this pair is being dropped. This cleaning step is applied only to the train set where 12 conflicting pairs were found as the test set contains no conflicting pairs.

The second cleaning step removes rows marked as inaudible (inaudible = True). This means that during the interview the audio quality was poor or unclear at that specific point so the annotator could not hear the answer properly making these entries useless for training the classifier. In the train set 45 such rows were found and dropped.

2.2. Text Preprocessing

The following preprocessing steps are applied to both the question and interview answer columns of the train and test sets:

- Lowercasing: All text is converted to lowercase.
- Contraction expansion: Shortened forms such as "don't" are expanded to their full forms (e.g. "do not") using the contractions library
- Em dash replacement: Em dashes (—) are replaced with a space

- Unicode removal: Special unicode characters are removed.
- Punctuation removal: All punctuation characters are replaced with spaces to eliminate noise
- Stopword removal: A customized stopword list is used to remove the stopwords from the text
- Whitespace normalization: Extra whitespace characters are removed from the text

Finally after text preprocessing any rows where the interview answer was reduced to zero words are removed as empty answers carry no useful information for classification. In the train set 4 such rows were found and removed while the test set contained no such rows

2.3. Analysis

The exploratory data analysis is performed on the train set and on the test set after the data cleaning and text preprocessing steps described in sections 2.1 and 2.2. The train set contains 3,375 samples and the test set contains 308 samples covering interviews from four U.S. presidents: Donald J. Trump, Barack Obama, George W. Bush and Joseph R. Biden.

Label Distribution
The dataset is significantly imbalanced across all three classes. In the train set, which contains 3,375 samples the Ambivalent class dominates with 2,000 samples (59.3%), followed by Clear Reply with 1,026 samples (30.4%) and Clear Non-Reply with only 349 samples (10.3%). The same imbalance pattern is observed in the test set, which contains 308 samples where Ambivalent accounts for 206 samples (66.9%), Clear Reply for 79 samples (25.6%) and Clear Non-Reply for only 23 samples (7.47%). This class imbalance is an important consideration for model training as a classifier that simply predicts Ambivalent for every sample would already achieve a high accuracy. This is why macro F1-score is used as the evaluation metric instead of accuracy since it treats all three classes equally regardless of their size.

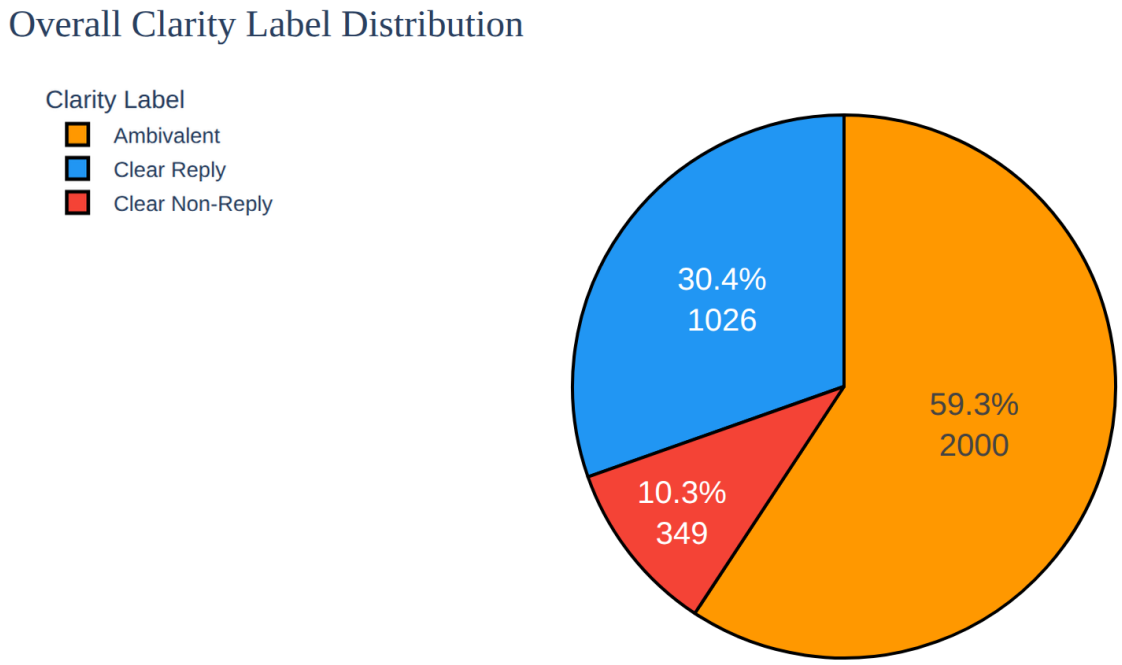


Figure 1: Label distribution in the training set

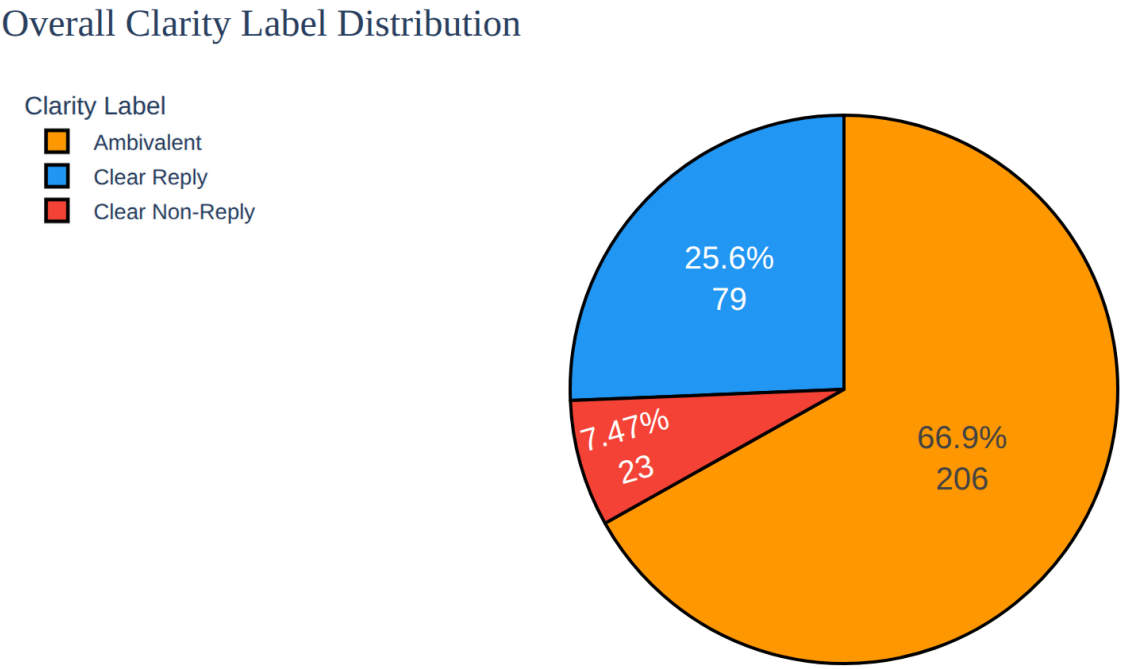


Figure 2: Label distribution in the test set

Label Distribution per President

As shown in Figure 3, Donald J. Trump has the highest total number of samples in the train set (1,310 representing 38.8% of the data), followed by Barack Obama (978 representing 29% of the data), George W. Bush (697 representing 20.7% of the data) and Joseph R. Biden (390 representing 11.6% of the data)

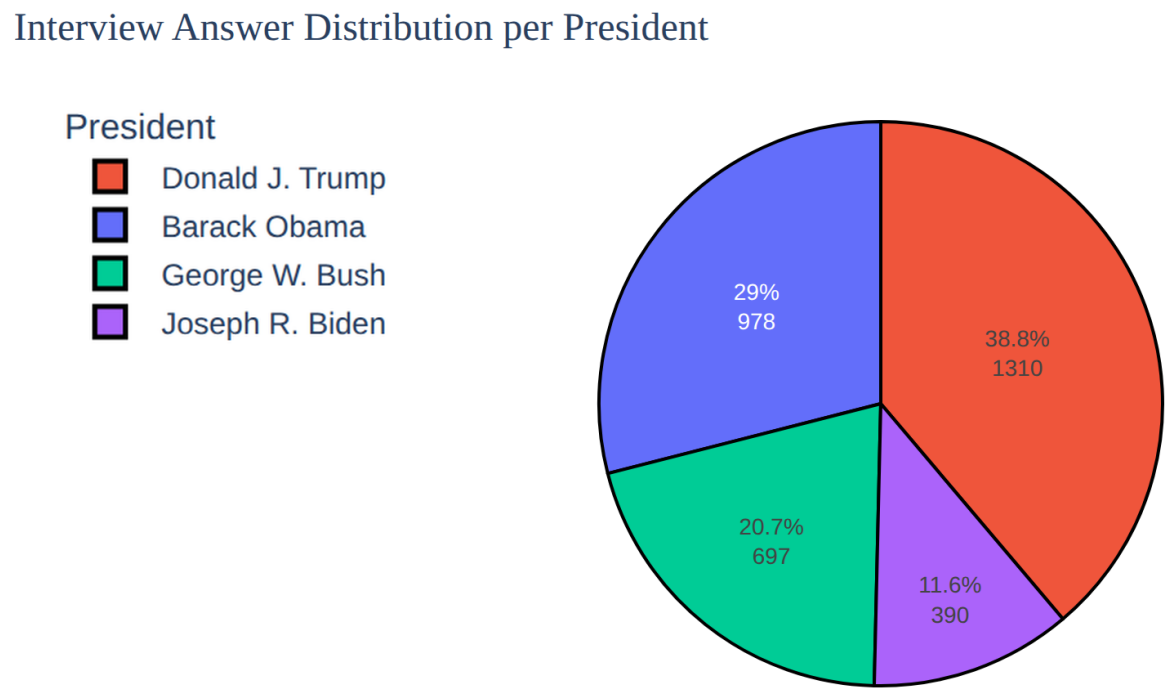


Figure 3: Label distribution per president in the training set

As shown in Figure 4 the Ambivalent class is consistently the dominant class across all four presidents. For Donald J. Trump the distribution is 731 Ambivalent, 426 Clear Reply and 153 Clear Non-Reply. For Barack Obama it is 669 Ambivalent, 215 Clear Reply and 94 Clear Non-Reply. For George W. Bush it is 397 Ambivalent, 238 Clear Reply and 62 Clear Non-Reply. Finally for Joseph R. Biden it is 203 Ambivalent, 147 Clear Reply and 40 Clear Non-Reply

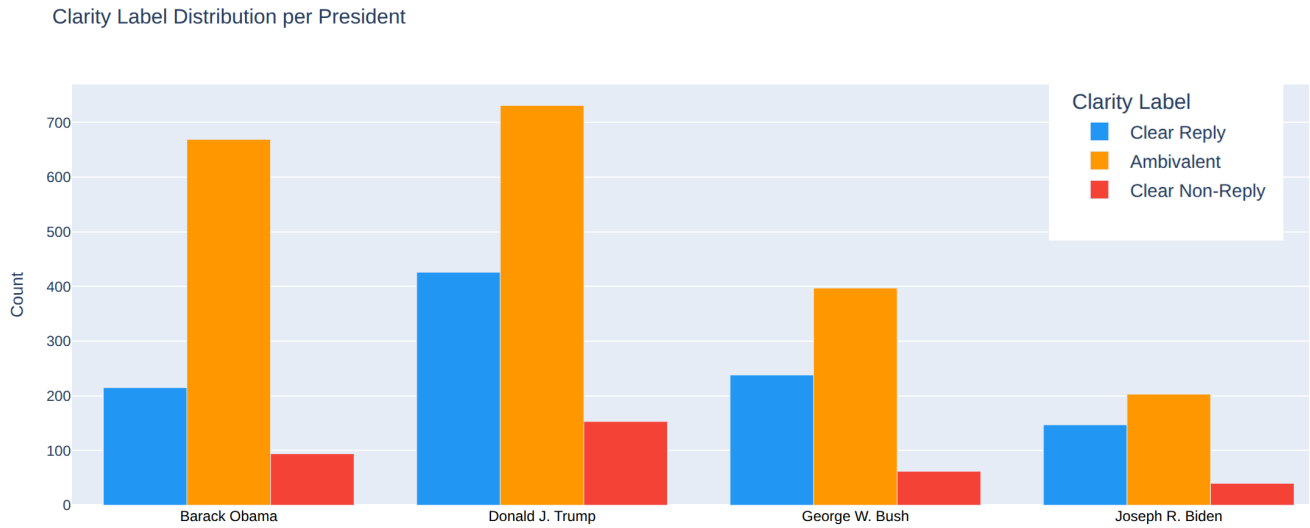


Figure 4: Label distribution by class for each president in the training set

Answer Length Statistics
The answer length statistics reveal meaningful differences across the three clarity labels. Ambivalent answers are on average the longest with a mean of 164.6 words and a median of 119.0 words suggesting that when presidents give ambivalent responses they tend to speak at length without fully committing to a clear answer. Clear Reply answers have an average of 134.6 words and a median of 96.5 words. Clear Non-Reply answers are notably shorter with an average of only 67.6 words and a very low median of 19.0 words indicating that when presidents choose not to answer a question they tend to do so with very few words rather than giving a long evasive response.

Label	Average Length	Median	Min	Max
Clear Reply	134.6	96.5	1	767
Ambivalent	164.6	119.0	1	1015
Clear Non-Reply	67.6	19.0	1	755

Table 1: Answer length statistics per label on the training set

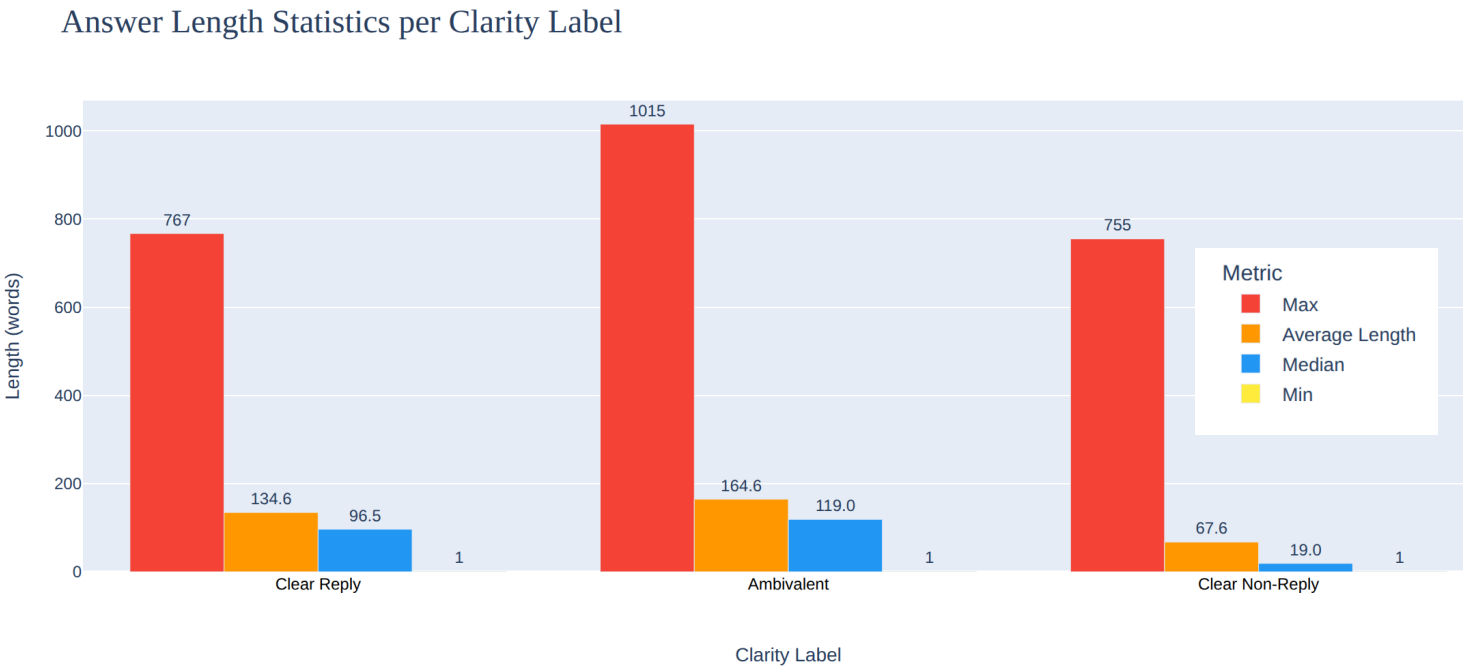


Figure 5: Answer length distribution by clarity label in the training set

Most Frequent Words per Class
Examining the top 100 most frequently used words per clarity label shows that the majority of words appear across all three classes. This means that we cannot identify words that are exclusive to a specific class. In other words we cannot identify whether a specific word appears mainly in only one class and therefore draw any meaningful

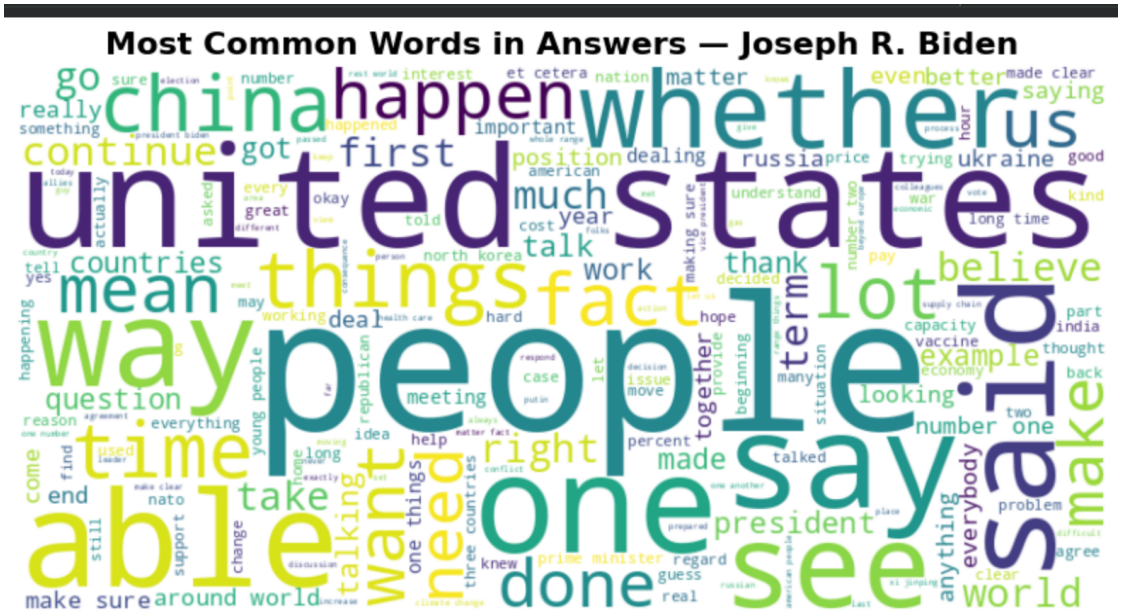
conclusions from it. So we cannot conclude that if a specific word appears in a president's answer, the answer most likely belongs to a particular class. For example the word "not" appears as the most frequent word in all three classes so we cannot say that if a president uses the word "not" in his answer that answer most likely belongs to a specific class.

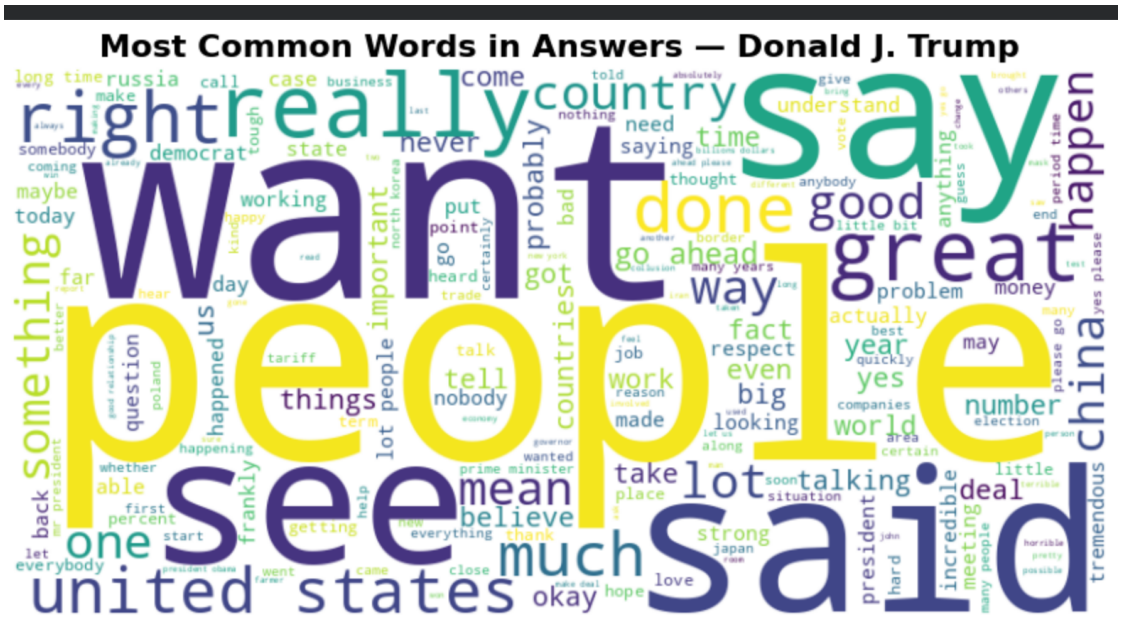
Top 100 frequent words per class (parallel view)

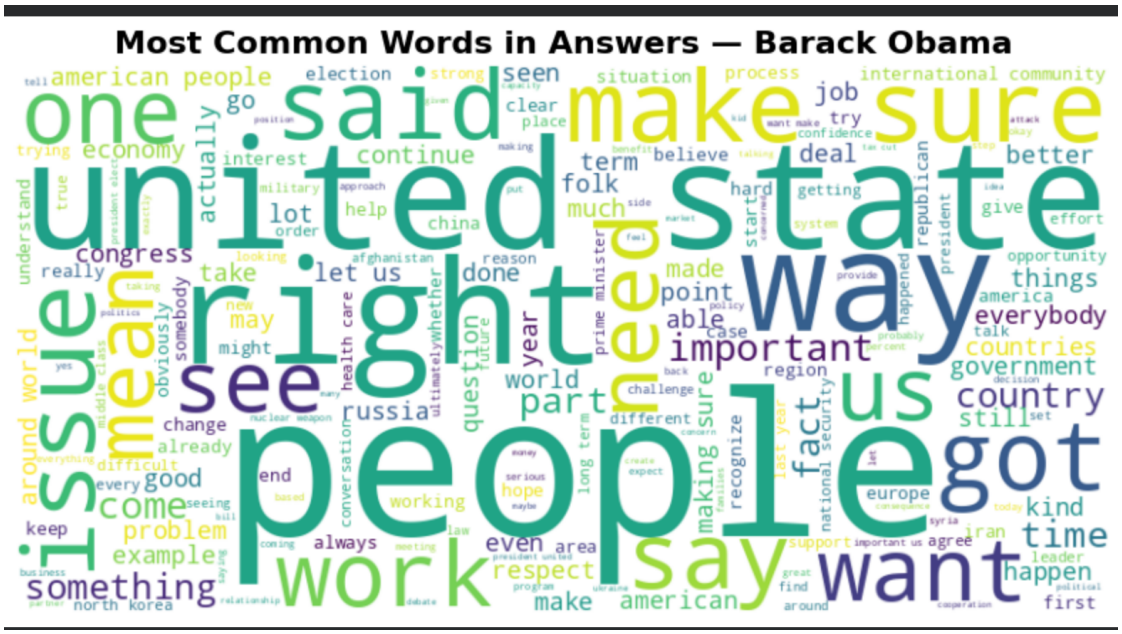
Rank	Clear Reply	Count	Ambivalent	Count	Clear Non-Reply	Count
1	not	3690	not	8620	not	833
2	but	2007	but	4804	but	380
3	what	1680	what	4130	what	366
4	people	1613	people	3432	very	248
5	very	1285	very	3140	people	234
6	if	1091	if	2619	would	189
7	would	1026	can	2461	if	189
8	can	985	would	2386	can	170
9	just	833	just	1844	just	143
10	one	800	us	1820	make	140
11	us	709	when	1757	want	137
12	get	691	one	1750	like	133
13	when	690	make	1712	get	119
14	more	681	who	1679	when	117
15	who	665	get	1589	lot	116
16	make	659	president	1529	who	115
17	want	646	want	1491	one	114
18	no	623	like	1488	question	113
19	like	621	states	1438	us	110
20	way	592	united	1405	time	103
21	president	583	more	1396	no	103
22	said	582	said	1388	say	95
23	time	537	way	1292	right	94
24	say	524	country	1277	sure	90
25	lot	523	how	1256	united	88
26	states	520	lot	1249	see	87
27	united	518	say	1240	work	87
28	world	508	time	1221	go	85
29	see	496	see	1173	states	85
30	how	495	where	1168	got	85
31	good	493	right	1164	where	84
32	which	491	world	1156	also	82
33	got	491	got	1153	how	80
34	country	487	work	1147	way	80
35	right	478	which	1138	good	80
36	where	460	good	1119	need	79
37	go	446	no	1107	which	76
38	things	445	sure	1100	world	74
39	yes	423	countries	1080	said	73
40	sure	415	also	1046	country	73
41	work	412	done	1009	much	72
42	also	405	go	984	more	71
43	countries	393	should	961	cannot	71
44	done	389	important	960	let	70
45	important	376	things	938	done	68
46	able	370	take	895	countries	68

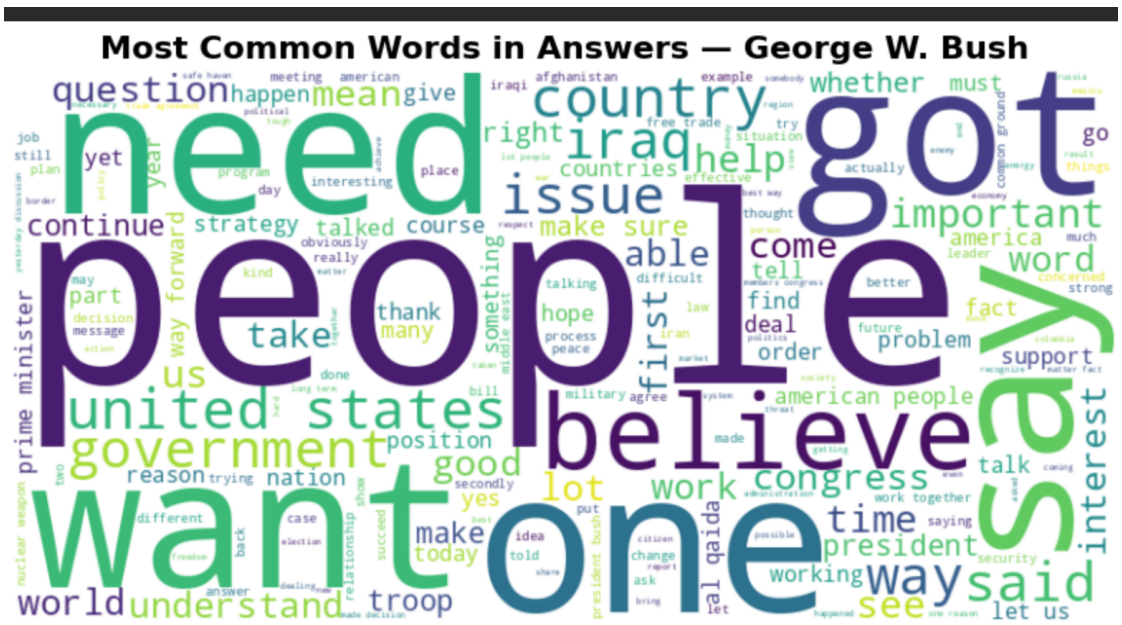
Rank	Clear Reply	Count	Ambivalent	Count	Clear Non-Reply	Count
47	should	358	made	843	back	68
48	why	344	able	817	thank	65
49	much	339	much	816	things	62
50	deal	335	government	801	part	60
51	made	332	great	786	really	60
52	take	330	first	774	important	58
53	first	324	security	756	able	58
54	need	322	many	749	american	56
55	believe	317	back	747	terms	56
56	part	311	american	739	may	56
57	years	310	part	735	take	56
58	great	310	need	733	president	55
59	question	307	believe	724	whether	55
60	issue	307	years	715	mean	54
61	could	303	something	711	russia	53
62	really	301	question	698	should	53
63	back	301	could	686	first	52
64	american	299	deal	682	place	51
65	government	296	why	675	new	50
66	mean	284	fact	656	china	50
67	china	282	really	653	talk	50
68	come	280	let	641	tell	49
69	working	279	issue	640	okay	49
70	fact	262	even	623	made	48
71	something	259	mean	618	deal	47
72	many	256	continue	612	could	46
73	even	255	china	597	looking	45
74	better	251	russia	596	put	45
75	cannot	251	only	595	making	45
76	help	250	economy	587	talking	44
77	making	245	long	581	continue	44
78	security	245	working	566	yes	44
79	continue	239	come	566	issue	44
80	congress	239	last	561	clear	43
81	last	230	making	555	security	43
82	place	230	help	555	different	43
83	long	229	cannot	555	ahead	43
84	economy	229	place	554	job	43
85	job	228	put	548	even	42
86	thank	221	america	537	believe	42
87	whether	221	trade	532	give	42
88	understand	216	whether	519	fact	42
89	america	215	yes	515	long	42
90	problem	215	thank	508	understand	42
91	let	214	issues	503	process	42
92	new	212	respect	502	respect	42
93	only	212	better	500	working	42
94	actually	211	same	498	something	41
95	international	210	may	495	strong	41
96	issues	205	most	494	great	40
97	most	205	new	488	problems	40
98	two	204	congress	481	most	40
99	around	201	job	477	still	40
100	different	200	understand	477	answer	39

Finally the following wordclouds illustrate the most frequently used words in the interview answers of each president in the training set









2.4. Data partitioning for train, test and validation

The dataset was obtained from HuggingFace and comes pre-divided into two splits: a train split and a test split.

The train split was further divided into a training set (85%) and a validation set (15%) using scikit-learn train_test_split function. The validation set was used to evaluate the model's performance during development without touching the test set.

Two important choices were made during the split:

- Stratification: The stratify=labels parameter was used to ensure that the class distribution of the three clarity labels (Clear Reply, Ambivalent, Clear Non-Reply) is

preserved proportionally in both the training and validation sets. This is particularly important since the dataset is imbalanced, with the Ambivalent class accounting for approximately 59.3% of the training data. Stratification ensures that all three classes (Clear Reply, Ambivalent and Clear Non-Reply) are adequately represented in both the training and validation sets.

- Random seed: A fixed random_state=42 was used to ensure reproducibility of the results.

2.5. Vectorization

Prior to vectorization the question and interview answer fields were concatenated into a single input string using the format: "question: [question text] answer: [answer text]". By combining both fields the model is provided with the full conversational context needed to classify the clarity of the response.

TF-IDF Vectorization Part

In the first approach the TF-IDF (Term Frequency–Inverse Document Frequency) representation was used via scikit-learn TfidfVectorizer. TF-IDF assigns a weight to each token that reflects how frequently it appears in a document relative to how common it is across all documents in the corpus. Tokens that appear frequently in a specific document but rarely across the corpus receive higher weights, making them more informative features.

One key configuration choice was made: sublinear TF scaling was applied as a fixed design decision. Instead of using raw term frequency the transformation $1 + \log(tf)$ was applied which reduces the effect of very frequent terms and prevents them from dominating the feature space.

Word2Vec Vectorization Part

The second approach uses Word2Vec embeddings trained from scratch on our dataset using the gensim library. Rather than relying on a pre-trained model such as Google News vectors we trained our own Word2Vec model so that the resulting vectors reflect the specific vocabulary and context of our political interview data. Unlike TF-IDF the Word2Vec assigns each token a dense vector of numbers that captures its meaning and its relationship to other words in the vocabulary. Words that appear in similar contexts end up with similar vectors making them more semantically informative features.

The following configuration choices were made:

- Architecture: The Skip-gram architecture (sg=1) was used which trains the model to predict surrounding context words given a target word.
- Vector size: Each token was represented as a 650-dimensional vector.
- Window size: A context window of 10 was used meaning the model considers 10 words on each side of the target word during training.
- Training epochs: The model was trained for 80 epochs.

- Minimum count: Tokens appearing fewer than 2 times in the corpus were ignored.
- Document representation: Each QA pair was encoded as the average of its token vectors. Out-of-vocabulary (OOV) words not seen during training were skipped.
- Reproducibility: A fixed seed=42 was used to fix the Word2Vec random initialization so results are the same every run. Setting workers=1 is required for the seed to work, as single thread ensures full reproducibility.

3. Algorithms and Experiments

3.1. TF-IDF + Logistic Regression Part

The experimental process began with a baseline model adapted from the lab code using manually set parameters: ngram_range=(1,2), C=0.5, sublinear_tf=True, class_weight='balanced', max_iter=1000 and random_state=42. This served as a reference point to evaluate whether further optimization could improve performance. However it quickly became clear that finding the best parameter combination manually through trial and error is impractical as the number of possible combinations grows rapidly. For this reason all subsequent experiments were conducted using GridSearchCV, which automatically searches all possible combinations of the provided parameters and returns the configuration that achieves the best macro F1 score using 5-fold cross-validation. The parameters searched by GridSearchCV were ngram_range, C, solver, class_weight and penalty while sublinear_tf=True, max_iter=1000 and random_state=42 were kept fixed across all experiments.

In **GridSearch Experiment 1** I used the method of "question: [question text] answer: [answer text]" concatenation format for the train and test set but without applying the custom stopword removal mechanism. GridSearchCV found the best parameters to be C=1.3, solver=lbfgs, penalty=l2, class_weight=balanced and ngram_range=(1,2), achieving a Cross-Validation Macro F1 of 0.5621, a Validation Macro F1 of 0.5277 and a Test Macro F1 of 0.4314.

In **GridSearch Experiment 2** I kept the same "question: [question text] answer: [answer text]" concatenation format for the train and test set, while applying the custom stopword removal mechanism. GridSearchCV found the best parameters to be C=0.9, solver=lbfgs, penalty=l2, class_weight=balanced and ngram_range=(1,2), achieving a Cross-Validation Macro F1 of 0.5555, a Validation Macro F1 of 0.5087 and a **Test Macro F1 of 0.4413. This was the best test score across all experiments.** The stopword removal helped the model focus on more meaningful content words rather than frequently occurring but uninformative words, which improved generalization to unseen data.

In **GridSearch Experiment 3** I replaced the "question: [question text] answer: [answer text]" concatenation format with the simpler method question + " " + answer, while applying the custom stopword removal mechanism. GridSearchCV found the best parameters to be C=0.8, solver=lbfgs, penalty=l2, class_weight=balanced and ngram_range=(1,2 achieving a Cross-Validation Macro F1 of 0.5432, a Validation Macro F1 of 0.5214 and

a Test Macro F1 of 0.4285. This experiment produced a lower Test Macro F1 score of 0.4285. This suggests that the "question: [question text] answer: [answer text]" concatenation format is important as it helps the model distinguish between the question and answer parts of the input. Without this distinction the model struggles to understand the relationship between the two parts.

In **GridSearch Experiment 4** I again replaced the "question: [question text] answer: [answer text]" concatenation format with the simpler method question + " " + answer, but without applying the custom stopword removal mechanism. GridSearchCV found the best parameters to be C=1.0, solver=lbfgs, penalty=l2, class_weight=balanced and ngram_range=(1,2), achieving a Cross-Validation Macro F1 of 0.5570, a Validation Macro F1 of 0.5208 and a Test Macro F1 of 0.4204. This experiment produced a lower Test Macro F1 score of 0.4204. This suggests that the "question: [question text] answer: [answer text]" concatenation format is important as it helps the model distinguish between the question and answer parts of the input. Without this distinction the model struggles to understand the relationship between the two parts. Also applying the custom stopword removal mechanism helped the model focus on more meaningful content words rather than frequently occurring but uninformative words, which improved generalization to unseen data.

In **GridSearch Experiment 5** I used the same "question: [question text] answer: [answer text]" concatenation format for the train and test set and applied the custom stopword removal mechanism for both sets, but I added an extra preprocessing step of dropping rows where the answer contained fewer than 5 words. GridSearchCV found the best parameters to be C=0.5, solver=lbfgs, penalty=l2, class_weight=balanced and ngram_range=(1,2), achieving a Cross-Validation Macro F1 of 0.4978, a Validation Macro F1 of 0.5607 and a Test Macro F1 of 0.4116. While the validation F1 increased significantly to 0.5607 (in GridSearch Experiment 2 we had 0.5087), the Test F1 dropped to 0.4116 (in GridSearch Experiment 2 we had 0.4413). This can be explained by the fact that rows with fewer than 5 words tend to contain more answers labeled as Clear Reply or Clear Non-Reply than Ambivalent, which is intuitive since short answers are easier to classify as clearly responsive or clearly non-responsive. By dropping these rows the already imbalanced dataset became even more imbalanced, as we were removing samples that predominantly belonged to the minority classes. This reduced the diversity of the training data causing the model to overfit to the remaining longer answers and generalize poorly to unseen data, given that short answers are still present in the unseen data.

In **GridSearch Experiments 7 and 8** I used the same "question: [question text] answer: [answer text]" concatenation format for the train and test set. Both experiments applied an extra preprocessing step of dropping rows where the answer contained fewer than 2 words, with the custom stopword removal mechanism applied in Experiment 7 and without it in Experiment 8. Both experiments produced identical results with a Validation Macro F1 of 0.5927 and a Test Macro F1 of 0.3924, which was the worst test score across all experiments.

In **GridSearch Experiment 9** I used the same setup as Experiment 2 but reduced the validation set size from 15% to 10%. GridSearchCV found the best parameters to be C=1.0, solver=lbfgs, penalty=l2, class_weight=balanced and ngram_range=(1,2), achieving a Cross-Validation Macro F1 of 0.5567, a Validation Macro F1 of 0.5308 and

a Test Macro F1 of 0.4335. The slightly lower test score compared to Experiment 2 confirms that a 15% validation split provides a more reliable evaluation signal during development than a 10% split.

Attempt	Concatenation Format	Stopword Removal	Drop Short Answers	Validation Size	Cross Val F1	Val F1	Test F1
GridSearch Experiment 1	question: [question text] answer: [answer text]	No	No	15%	0.5621	0.5277	0.4314
GridSearch Experiment 2 (Best)	**question: [question text] answer: [answer text]**	**Yes**	**No**	**15%**	**0.5555**	**0.5087**	**0.4436**
GridSearch Experiment 3	question_text + " " + answer_text	Yes	No	15%	0.5432	0.5214	0.4285
GridSearch Experiment 4	question_text + " " + answer_text	No	No	15%	0.5570	0.5208	0.4204
GridSearch Experiment 5	question: [question text] answer: [answer text]	Yes	< 5 words	15%	0.4978	0.5607	0.4116
GridSearch Experiment 6	question: [question text] answer: [answer text]	No	< 5 words	15%	0.5339	0.5942	0.4283
GridSearch Experiment 7	question: [question text] answer: [answer text]	Yes	< 2 words	15%	0.5272	0.5927	0.3924
GridSearch Experiment 8	question: [question text] answer: [answer text]	No	< 2 words	15%	0.5272	0.5927	0.3924
GridSearch Experiment 9	question: [question text] answer: [answer text]	Yes	No	10%	0.5567	0.5308	0.4335

3.1.1. Table of trials TF-IDF + Logistic Regression Part.

3.2. Word2Vec + Logistic Regression Part

Unlike TF-IDF the Word2Vec model involves two levels of hyperparameters to tune:

- The Word2Vec parameters (vector_size, window, epochs) were tuned manually through a systematic nested loop structure as shown below:

```
for window in [10]:
    for vector_size in [650]:
        for epochs in [80]:
```

where each combination of values was tested exhaustively. For each combination, a new Word2Vec model was trained from scratch and the documents were vectorized using the resulting embeddings.

- The Logistic Regression parameters were tuned automatically using GridSearchCV, which searches all possible combinations of the provided parameters and returns the configuration that achieves the best macro F1 score using 5-fold cross-validation. The parameters searched by GridSearchCV were C, solver, penalty and class_weight, while max_iter=10000 and random_state=42 were kept fixed across all experiments

The combination of Word2Vec and Logistic Regression parameters that achieved the highest Validation Macro F1 score was then selected as the final configuration.

First Attempts & Early Observations

Our initial approach was to train a Word2Vec model from scratch on the training data and use the resulting document vectors as input to a Logistic Regression classifier. During the first runs we observed that rerunning the same parameter configuration produced different validation scores each time. This revealed a non-deterministic behavior in Word2Vec training. So the Word2Vec requires both seed=42 and workers=1 to guarantee full reproducibility, since multithreaded execution introduces randomness that cannot be controlled by the seed alone. This fix was applied from the second attempt onwards ensuring that all subsequent results were directly comparable.

A second key modification was switching Word2Vec from its default CBOW (Continuous Bag of Words) mode to Skip-gram by setting sg=1. Skip-gram is generally better suited for smaller datasets as it learns to predict the surrounding context from a single word, producing richer representations for less frequent vocabulary. This change had an immediate positive impact on validation Macro F1 confirming it was the right direction to follow.

3.2.1. Table of trials Word2Vec + Logistic Regression Part.

Vector Size	Epochs	Val Macro F1
100	5	0.3946
100	10	0.4333
100	50	0.4119
300	5	0.4039
300	10	0.4382
300	20	0.4405
300	25	0.4441
300	30	0.4503
350	5	0.4146
350	10	0.4218
350	15	0.4391
350	20	0.4613
350	25	0.4559
350	30	0.4500

Vector Size	Epochs	Val Macro F1
350	35	0.4584
400	15	0.4353
400	20	0.4560
400	25	0.4575
400	30	0.4861
400	35	0.4729
400	40	0.4798
450	25	0.4581
450	30	0.4508
450	35	0.4381
450	40	0.4378
500	25	0.4511
500	30	0.4602
500	35	0.4554
500	40	0.4526
600	25	0.4555
600	30	0.4562
600	35	0.4692
600	40	0.4735
600	45	0.4853
600	50	0.4621
600	55	0.4653
650	25	0.4615
650	30	0.4673
650	35	0.4740
650	40	0.4499
650	45	0.4550
650	50	0.4591
650	55	0.4510
650	60	0.4534
650	65	0.4611
650	70	0.4544
650	75	0.4559
650	80	0.4689
700	25	0.4690
700	30	0.4733
700	35	0.4883
700	40	0.4633
700	45	0.4933
700	50	0.4975
700	55	0.4961
700	60	0.4824
700	65	0.4639
700	70	0.4587
750	25	0.4451
750	30	0.4652
750	35	0.4672
750	40	0.4480
750	45	0.4588
750	50	0.4613
750	55	0.4950
750	60	0.4953

Vector Size	Epochs	Val Macro F1
750	65	0.4945
750	70	0.4515
800	25	0.4628
800	30	0.4699
800	35	0.4720
800	40	0.4458
800	45	0.4507
800	50	0.4523
800	55	0.4477

Table 3: Word2Vec hyperparameter search results

Table 3 presents the results of the Word2Vec hyperparameter search, showing the Validation Macro F1 score obtained for each combination of vector_size and epochs tested.

3.2.2. Analysis of Experimental Results.

To better understand the results obtained from the experiments, we created several visualizations to help interpret and compare the Validation Macro F1 scores across the different vector_size and epochs configurations.

Heatmap

A Heatmap displays a matrix where each cell corresponds to a specific combination of vector_size (y-axis) and epochs (x-axis). The color of each cell encodes the Validation Macro F1 score for that configuration, with brighter colors indicating higher scores and darker colors indicating lower scores. Empty cells correspond to combinations that were not tested during the search.



Figure 6: Heatmap of Validation Macro F1 by vector_size and epochs

From the above heatmap we can draw the following conclusions:

1. The bright region (best performance) is clearly concentrated around vector_size=700–750 and epochs=45–65, with the absolute peak at vector_size=700, epochs=50 (F1=0.4975).
2. The dark/cold region (worst performance) is the bottom left, i.e. vector sizes 100 and 300, which never exceed 0.4503 regardless of epochs, showing these sizes are simply too small for this task.
3. Vector sizes 450–500 remain flat and mediocre (around 0.45–0.46 never breaking 0.46) across all epochs tested, never breaking through despite more training.
4. Vector size 800 shows a visible darkening as epochs increase beyond 35, suggesting the model starts to overfit or degrade with too many training steps at this size.

Grouped Bar Chart

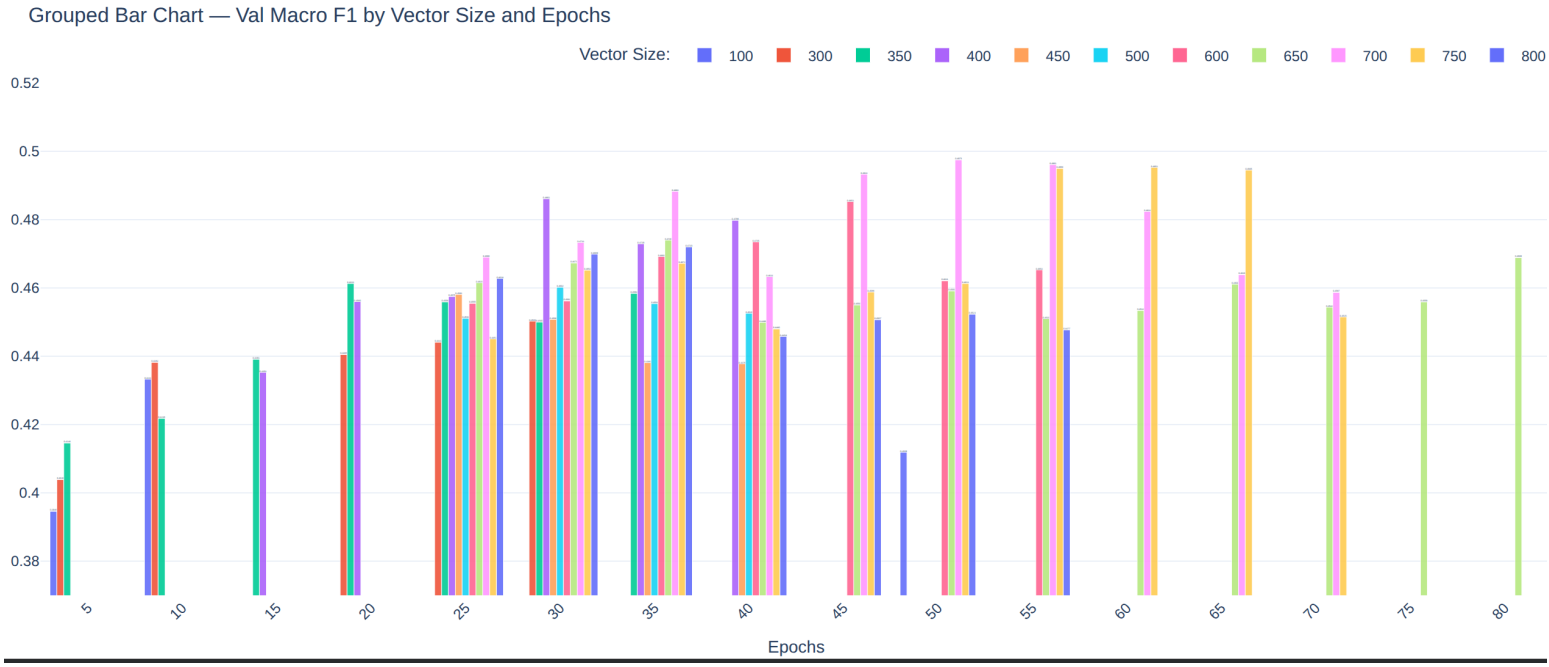


Figure 7: Grouped Bar Chart of Validation Macro F1 across vector_size and epochs

From the above grouped bar chart we can draw the following conclusions:

1. At low epochs (5–20) all vector sizes cluster at lower F1 values with no single size standing out significantly i.e. all bars are short and similar in height.
2. Vector size 700 (pink bars) consistently produces some of the tallest bars from epoch 25 onwards, peaking clearly at epoch 50 (0.4975) and remaining strong through epoch 55 (0.4961).
3. Vector size 750 (orange bars) starts modestly but surges at epochs 55–65, becoming competitive with 700 at those later epochs.
4. Vector sizes 100 and 300 only appear at very early epochs and have the shortest bars confirming they are the weakest configurations.
5. At each epoch value it is easy to see which vector size dominates i.e. larger sizes (600–750) generally produce taller bars than smaller sizes (100–500) at the same epoch.
6. No single vector size dominates at all epochs i.e. the best bar at each epoch position changes, showing that vector_size and epochs must be tuned together rather than independently.

Conclusion

It is important to note that the scores discussed in the tables and visualizations above are Validation Macro F1 scores, used for model selection during experimentation. The following scores refer to the Kaggle competition test scores.

Through a series of experiments we found that vector_size=650 with epochs=80 achieved the best Kaggle test score of 0.58, while vector_size=600 with epochs=40 achieved the second best score of 0.56. This is consistent with the graphs as both configurations showed solid validation F1 scores above 0.46. Interestingly vector_size=700 and 750, which achieved the highest validation Macro F1 scores (peaking at 0.4975), only reached Kaggle test scores in the range of 0.54–0.55. Although these scores are not poor, they fall below those of the 600 and 650 configurations, suggesting a slight degree of overfitting to the validation set.

Explaining the Results

Why do small vector sizes (100–300) perform poorly ?

A vector size of 100 or 300 means each word is represented by only 100 or 300 numbers. This is not enough dimensions to capture the complexity and nuances of the vocabulary in our dataset. With too few dimensions, different words end up with very similar vectors, making it harder for the Logistic Regression classifier to distinguish between the three classes.

Why does performance peak around vector_size=700 ?

As vector_size increases from 100 to 700 the model gains more capacity to encode meaningful differences between words. However beyond 700 performance starts to drop slightly. At vector_size=750 and 800 the scores never consistently beat 700. This suggests that beyond a certain point the extra dimensions do not add useful information but instead introduce noise making the classification task harder rather than easier.

Why do too many epochs sometimes hurt ?

Looking at vector_size=800, for example, performance drops as epochs increase beyond 35. This is because with too many training steps the model starts overfitting to the specific patterns in the training vocabulary, producing vectors that are too specialized and do not generalize well.

Why must vector_size and epochs be tuned together ?

The sweet spot for epochs depends on the vector size, larger vectors need more epochs to converge properly. For example vector_size=700 peaks at epochs=50, while vector_size=750 needs epochs=60–65 to reach its best. This is why no single epoch value works best across all vector sizes.

Pre-trained vs From-Scratch Word2Vec

As an additional experiment we tested using a pre-trained Google News Word2Vec model instead of training one from scratch on our data. Importantly the exact same GridSearchCV Logistic Regression pipeline was used in both cases so that we can compare them in an objective way, with the only difference being the source of the word vectors. Using the Google News pre-trained vectors to represent our documents, we obtained a Validation Macro F1 of 0.4671 and a Test Macro F1 of 0.4299 with best parameters found by the GridSearchCV function C=0.8, class_weight=balanced,

penalty=l2, solver=saga.

In contrast by training Word2Vec from scratch on our own dataset, our best experiment achieved a Validation Macro F1 of 0.4689 and a Test Macro F1 of 0.4775, a notable improvement of +0.0476 on the test set, with best parameters found by the GridSearchCV function C=0.7, class_weight=balanced, penalty=l2, solver=liblinear. Interestingly the validation scores are very close to each other (0.4689 vs 0.4671), but the test scores differ significantly. This result can be explained by two key advantages of training from scratch. First the Google News model uses fixed hyperparameters, i.e. a fixed vector size of 300, a fixed window size and a fixed number of training epochs, none of which can be tuned, whereas our experiments showed that vector_size=650, window=10 and epochs=80 produced the best results for our data. Second the Google News model was trained on formal journalistic text, which is very different in style and vocabulary from the conversational, informal language present in our dataset. By training from scratch the model learns word vectors that are specifically tailored to the vocabulary and context of our data, allowing it to capture the subtle linguistic differences between the three classes much more effectively.

This experiment confirmed that for domain-specific tasks with specialized vocabulary, training Word2Vec from scratch on the target dataset is preferable to using a pre-trained model.

3.3. Hyper-parameter tuning

Parameter Search Setup

To avoid the impracticality of manual trial-and-error hyper-parameter tuning was performed using `GridSearchCV` with 5-fold cross-validation, which automatically searches all parameter combinations and selects the one with the best Macro F1 score. This approach was applied consistently to both the TF-IDF and Word2Vec models, ensuring a fair and systematic comparison.

Two hyperparameters were kept fixed across all experiments for both models:

- `class_weight='balanced'`: was kept fixed because the dataset is imbalanced across the three clarity labels, without it the model would favor the majority class and underperform on minority classes.
- `random_state=42`: kept fixed to ensure reproducibility across all experiments.

3.3.1. TF-IDF + Logistic Regression.

Parameters Searched

The parameters searched by `GridSearchCV` for the TF-IDF model were:

- `ngram_range`: `(1,1)` vs `(1,2)`
- `C`: `[0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.3, 1.5]`
- `solver`: `[lbfgs, liblinear, saga]`

- `penalty: [L1, L2]`

While `sublinear_tf=True` and `max_iter=1000` were kept fixed across all experiments. In total `GridSearchCV` evaluated 90 parameter combinations with 5-fold cross-validation, resulting in 450 fits.

Best Parameters Found

The best configuration found was C=0.9, solver=lbfgs, penalty=l2, ngram_range=(1,2) (i.e. GridSearch Experiment 2) achieving a Cross-Validation Macro F1 of 0.5555. Bigrams `(1,2)` outperformed unigrams `(1,1)` because they capture phrase-level patterns such as "not clear" or "do not", which are important for clarity classification. The `l2` penalty was selected over `l1` because it distributes regularization across all features rather than forcing sparsity, which is more suitable for TF-IDF where many features contribute weakly but collectively. A value of `C=0.9`, close to 1.0, indicates relatively low regularization allowing the model to fit the training data without overly penalizing feature weights. The `lbfgs` solver was selected over `liblinear` and `saga` because it is well suited for multiclass classification, making it more appropriate for this three-class problem. `liblinear` uses a one-vs-rest strategy, while `saga` is a stochastic optimization method designed for very large datasets and was less efficient at this scale. It is also important that these best parameters were found together with the preprocessing choices of Experiment 2 for the TF-IDF part. Specifically the format "question: [question text] answer: [answer text]" helped the model distinguish question and answer content, custom stopword removal emphasized more informative words and short answers were kept because they mostly belonged to minority classes (*Clear Reply* and *Clear Non-Reply*), removing them would have worsened class imbalance. These preprocessing choices are described in Section 3.1 for the TF-IDF part.

Underfitting / Overfitting Analysis

As shown in the learning curves (Section 3.5.2) there is a significant and persistent gap between training F1 (approximately 0.89–0.98) and validation F1 (approximately 0.34–0.51), indicating overfitting. Although `GridSearchCV` selected `C=0.9` as the best regularization strength, the model still memorizes training patterns and struggles to generalize to unseen data. The two curves do not converge even when using 90% of training data, suggesting that overfitting is not only a data-size issue but also a limitation of TF-IDF, which relies on exact lexical matches and cannot fully capture deeper semantic relations in political language.

Evaluating the best model on the validation set produced a validation macro F1-score of 0.5087 and a test macro F1-score of 0.4413. The gap between cross-validation macro F1-score (0.5555) and test macro F1-score (0.4413) further supports the presence of overfitting. This is a common challenge in imbalanced NLP classification tasks, where minority classes are harder to generalize.

3.3.2. Word2Vec + Logistic Regression.

Parameters Searched

The parameters searched by `GridSearchCV` for the Word2Vec model were:

- `C: [0.3, 0.5, 0.7, 1.0]`

- `solver`: `[lbfgs, liblinear, saga]` for L2, `[liblinear, saga]` for L1
- `penalty`: `[L1, L2]`

While `max_iter=10000` was kept fixed across all experiments. In total `GridSearchCV` evaluated 20 parameter combinations with 5-fold cross-validation, resulting in 100 fits.

Best Parameters Found

The best configuration found was C=0.7, solver=liblinear, penalty=l2, achieving a Cross-Validation Macro F1 of 0.4829 and a Validation Macro F1 of 0.4689. The `l2` penalty was selected over `l1` because Word2Vec document vectors are dense. Every dimension carries information, so forcing sparsity with `l1` would discard useful signal. The `l2` penalty instead distributes regularization smoothly across all dimensions, which is more appropriate for dense vector representations. A value of `C=0.7`, slightly below 1.0, indicates moderate regularization, preventing the model from overfitting to the training vectors. The `liblinear` solver was selected over `lbfgs` and `saga` because Word2Vec produces dense continuous vectors rather than sparse features and `liblinear`'s coordinate descent optimization handles dense representations of this dimensionality more efficiently, achieving the best cross-validation score among all solvers tested.

Underfitting / Overfitting Analysis

As shown in the learning curves (Section 3.5.3), there is a persistent gap between training F1 (0.64–0.78) and validation F1 (0.40–0.45), indicating overfitting. Although `GridSearchCV` selected `C=0.7` as the best regularization strength, the model still memorizes training patterns and struggles to generalize to unseen data. The two curves do not converge even when using 90% of training data, suggesting that overfitting is not only a data-size issue but also a limitation of the document representation itself, since averaging word vectors into a single document vector loses word order and local context information.

Evaluating the best model on the validation set produced a Validation Macro F1 of 0.4689 and a Test Macro F1 of 0.4775. The gap between the Cross-Validation Macro F1 (0.4829) and the Test Macro F1 (0.4775) further supports the presence of overfitting. However compared to the TF-IDF model, the gap between training and validation F1 is noticeably smaller, suggesting that Word2Vec generalizes better due to its ability to capture semantic relationships between words rather than relying on exact lexical matches and because it was trained from scratch on our own dataset, allowing it to learn linguistic representations specifically tailored to the vocabulary and context of our data, rather than relying on pre-trained embeddings such as the Google News model.

3.4. Optimization techniques

Common Techniques

GridSearchCV

The primary optimization framework used in this project was GridSearchCV from the scikit-learn library. Rather than manually tuning parameters through trial and error

GridSearchCV automatically evaluates all possible combinations of the provided parameter grid. In this project it was configured with 5-fold cross-validation and macro F1 as the scoring metric. Additionally n_jobs=-1 was set to use all available CPU cores in parallel to speed up the search.

Cross-Validation

Five-fold cross-validation was used as part of the GridSearchCV process to get a more reliable evaluation of model performance during parameter search. Instead of evaluating each parameter combination on a single train/validation split, the data is split into five folds and the model is trained and evaluated five times, each time using a different fold as the validation set. The final score is the average across all five folds.

Regularization

L1 and L2 regularization were both explored through GridSearchCV, along with different values of C which controls regularization strength (i.e. a smaller C means stronger regularization that penalizes large weights more heavily to prevent overfitting while a larger `C` means weaker regularization that allows the model to fit the training data more freely). By penalizing large weights regularization prevents the model from memorizing specific patterns in the training data that do not generalize to unseen data, which is a common problem in NLP classification tasks

Class Imbalance Handling

The dataset is significantly imbalanced across the three clarity labels: Ambivalent is the dominant class with 2000 samples (59.3%), followed by Clear Reply with 1026 samples (30.4%) and Clear Non-Reply being the minority class with only 349 samples (10.3%). To address this class_weight='balanced' was kept fixed across all experiments. This setting automatically adjusts the weights of each class in the loss function inversely proportional to their frequency, giving higher importance to minority classes during training. Without this setting the model would be biased toward predicting the majority class, resulting in poor macro F1 performance on the minority classes.

TF-IDF Specific Techniques

Solver Algorithms

In Logistic Regression, the solver is the optimization algorithm used to find the best model weights by minimizing the loss function. Three solvers were explored through GridSearchCV: lbfgs, liblinear and saga. lbfgs is well suited for multiclass classification problems, liblinear uses a one-vs-rest strategy which is less natural for multiclass tasks and saga is a stochastic gradient descent variant designed for very large datasets. lbfgs was consistently selected as the best solver across all TF-IDF experiments, which is expected given the nature of this problem, i.e. a three class classification task with a medium-sized dataset where lbfgs is more appropriate. In total 450 fits were performed (90 parameter combinations * 5 folds).

Regularization Results

L2 regularization was consistently selected as the best option across all TF-IDF

experiments as it distributes the penalty across all features rather than forcing sparsity, which is more suitable for TF-IDF representations where many features contribute weakly but collectively. C=0.9 was found to be the best value indicating relatively low regularization that allows the model to fit the training data well without overly penalizing the weights. Despite this as discussed in Section 3.3.1 the model still showed signs of overfitting, suggesting that regularization alone is not sufficient to bridge the gap between training and validation performance.

Together these optimization techniques formed a comprehensive strategy to maximize Macro F1 performance on this imbalanced multiclass classification task for the TF-IDF model.

Word2Vec Specific Techniques

Solver Algorithm

For the Word2Vec model three solvers were also explored through GridSearchCV: lbfgs, liblinear and saga. Unlike the TF-IDF model where lbfgs performed best here liblinear achieved the highest cross-validation score. This can be attributed to the nature of the input i.e. Word2Vec produces dense continuous vectors rather than sparse TF-IDF features and liblinear's coordinate descent optimization handles dense representations of this dimensionality more efficiently. In total 100 fits were performed (20 parameter combinations * 5 folds).

Regularization Results

L2 regularization was consistently selected as the best option across all Word2Vec experiments as it distributes the regularization smoothly across all dimensions, which is more appropriate for dense vector representations where every dimension carries information. C=0.7, slightly below 1.0, was found to be the best value indicating moderate regularization that prevents the model from overfitting to the training vectors. Despite this as discussed in Section 3.3.2 the model still showed signs of overfitting suggesting that regularization alone is not sufficient to bridge the gap between training and validation performance.

Skip-gram Training

By default Word2Vec uses the CBOW (Continuous Bag of Words) training algorithm. In this project Skip-gram was used instead by setting sg=1. Skip-gram learns to predict the surrounding context from a single word, which produces richer and more meaningful representations for less frequent vocabulary. This change had an immediate positive impact on validation Macro F1 confirming it was the right direction to follow.

Minimum Word Count

`min_count=2` was set to ignore words that appear only once in the training data. Words that appear very rarely do not have enough context to learn meaningful vector representations, so excluding them reduces noise in the model.

Reproducibility Fix

As discussed in Section 3.2, seed=42 and workers=1 were set to guarantee full reproducibility of the Word2Vec training process, since multithreaded execution introduces randomness that cannot be controlled by the seed alone.

Together these optimization techniques formed a comprehensive strategy to maximize Macro F1 performance on this imbalanced multicast classification task for the Word2Vec model.

3.5. Evaluation

Model performance was evaluated using four standard classification metrics: accuracy, precision, recall and F1-score.

> Accuracy measures the overall proportion of correctly classified samples.

> Precision measures how many of the samples predicted as a given class were actually that class.

> Recall measures how many of the actual samples of a given class were correctly identified by the model.

F1-score is the harmonic mean of precision and recall, providing a single balanced metric that accounts for both.

Since the dataset is significantly imbalanced across the three clarity labels (Ambivalent: 59.3%, Clear Reply: 30.4%, Clear Non-Reply: 10.3%), accuracy alone would be misleading, as a model that always predicts the majority class would still achieve a high accuracy score without actually learning to distinguish between classes. For this reason macro F1 was chosen as the primary evaluation metric, as it computes the F1-score independently for each class and averages them equally regardless of class size, giving equal importance to minority classes. This is also the official evaluation metric used in the Kaggle competition for this assignment.

All metrics were computed using scikit-learn's classification_report function, which provides per-class precision, recall and F1-score.

TF-IDF + Logistic Regression Part

Table 4 presents the per-class results of the best model (GridSearch Experiment 2) evaluated on the test set

Class	Precision	Recall	F1-score	Support
Ambivalent	0.70	0.78	0.74	206
Clear Non-Reply	0.35	0.26	0.30	23
Clear Reply	0.32	0.25	0.28	79
Accuracy			0.61	308
Macro avg	0.46	0.43	0.44	308

Table 4:

The results show that the model performs best on the Ambivalent class achieving an F1-score of 0.74, which is expected given that it is the dominant class in the dataset. Clear Non-Reply achieves a low F1-score of 0.30, while Clear Reply also remains weak with an F1-score of 0.28. Both minority classes score significantly lower than Ambivalent, confirming that the model struggles to generalize to less frequent classes despite the use of class_weight='balanced'. The overall Test Macro F1 of 0.4413 reflects this imbalance.

Word2Vec + Logistic Regression Part

Table 5 presents the per-class results of the best Word2Vec model (vector_size=650, window=10, epochs=80) evaluated on the test set.

Class	Precision	Recall	F1-score	Support
Ambivalent	0.71	0.84	0.77	206
Clear Non-Reply	0.41	0.39	0.40	23
Clear Reply	0.36	0.20	0.26	79
Accuracy			0.64	308
Macro avg	0.50	0.48	0.48	308

Table 5:

The results show that the model performs best on the Ambivalent class achieving an F1-score of 0.77, which is expected given that it is the dominant class in the dataset. Clear Non-Reply achieves a moderate F1-score of 0.40, while Clear Reply remains the weakest class with an F1-score of 0.26. The overall Test Macro F1 of 0.4775 is higher than the TF-IDF model, confirming that Word2Vec generalizes better across all three classes, particularly on the minority classes.

Comparison TF-IDF vs Word2Vec

Comparing the two models Word2Vec significantly outperforms TF-IDF on the Clear Non-Reply minority class (0.40 vs 0.30) and achieves similar performance on Clear Reply (0.26 vs 0.28) and Ambivalent (0.77 vs 0.74). The overall Test Macro F1 confirms the advantage of Word2Vec achieving 0.4775 compared to 0.4413 for TF-IDF. This suggests that semantic word representations learned from the dataset itself are more effective than exact lexical matching for this classification task, particularly for the minority classes that are harder to generalize. This can be attributed to the fact that Word2Vec was trained from scratch on our own dataset, allowing it to learn linguistic representations specifically tailored to the vocabulary and context of our data rather than relying on pre-trained embeddings such as the Google News model.

3.5.1. Learning Curve.

A learning curve plots model performance as a function of the amount of training data used, allowing us to assess whether the model suffers from overfitting or underfitting. Specifically it tracks both the training and validation Macro F1 scores as the model is trained on progressively larger subsets of the training data, from 10% up to 90% in steps of 10%.

3.5.2. TF-IDF + Logistic Regression Part.

The learning curve was generated using the best model found by GridSearchCV (C=0.9, solver=lbfgs, penalty=L2, class_weight=balanced, ngram_range=(1,2)), trained 9 times on progressively larger subsets of the training data. At each step both the training and validation Macro F1 scores were recorded and plotted.

As shown in Figure 8, the training F1 starts very high at 0.98 with only 10% of the data and gradually decreases to 0.89 at 90%, while the validation F1 starts low at 0.34 and slowly improves to 0.51. The two curves never converge maintaining a large and persistent gap throughout all training sizes. The gap between the Cross-Validation Macro F1 (0.5555) and the Test Macro F1 (0.4413) further confirms that the model suffers from a degree of overfitting. This is a common challenge in NLP classification tasks, particularly when the dataset is imbalanced and the minority classes are harder to generalize. The dataset is significantly imbalanced across the three clarity labels: Ambivalent is the dominant class with 2000 samples (59.3%), followed by Clear Reply with 1026 samples (30.4%) and Clear Non-Reply being the minority class with only 349 samples (10.3%).

Notably the validation F1 shows diminishing returns beyond 60% of training data (0.49 $\rightarrow$ 0.51), suggesting that simply adding more training data is unlikely to close the gap. This indicates that the overfitting is not primarily a data quantity problem but rather a fundamental limitation of the TF-IDF representation itself, which relies on exact word matches and cannot capture the deeper semantic relationships present in political language.

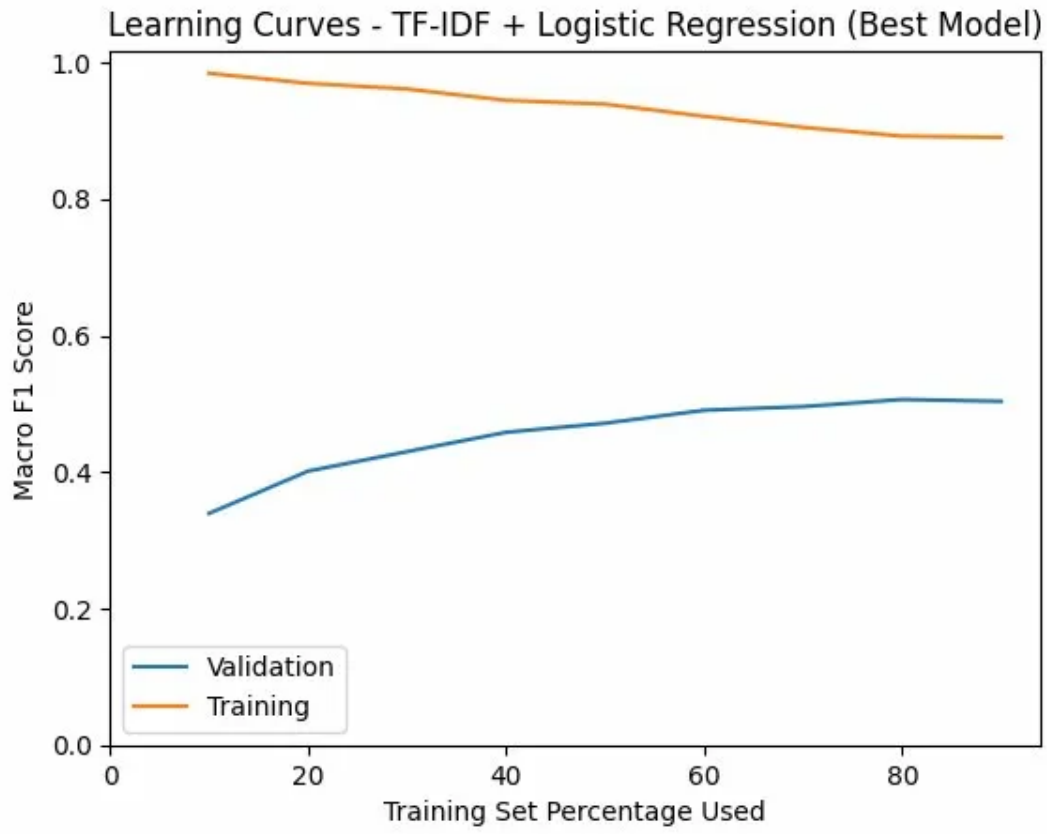


Figure 8: Learning Curves — TF-IDF + Logistic Regression (Best GridSearch Model)

Training %	Train F1	Validation F1
10%	0.98	0.34
20%	0.97	0.40
30%	0.96	0.43
40%	0.94	0.46
50%	0.94	0.47
60%	0.92	0.49
70%	0.90	0.50
80%	0.89	0.51
90%	0.89	0.50

Table 6: Training and Validation Macro F1 Scores per Training Subset Size — TF-IDF + Logistic Regression (Best GridSearch Model)

3.5.3. Word2Vec + Logistic Regression Part.

The learning curve was generated using the best model found (vector_size=650, window=10, epochs=80, C=0.7, solver=liblinear, penalty=l2, class_weight=balanced), trained 9 times on progressively larger subsets of the training data. At each step both the training and validation Macro F1 scores were recorded and plotted.

As shown in Figure 9, the training F1 starts at 0.78 with only 10% of the data and gradually decreases to 0.65 at 90%, while the validation F1 starts at 0.41 and slowly improves to 0.45. The two curves never converge maintaining a persistent gap throughout all training sizes. The gap between the Cross-Validation Macro F1 (0.4829) and the Test Macro F1 (0.4775) further confirms that the model suffers from a degree of overfitting. This is a common challenge in NLP classification tasks, particularly when the dataset is imbalanced and the minority classes are harder to generalize. The dataset is significantly imbalanced across the three clarity labels: Ambivalent is the dominant class with 2000 samples (59.3%), followed by Clear Reply with 1026 samples (30.4%) and Clear Non-Reply being the minority class with only 349 samples (10.3%).

Notably the validation F1 shows very slow improvement beyond 60% of training data (0.43 $\rightarrow$ 0.45), suggesting that simply adding more training data is unlikely to close the gap significantly. This indicates that the overfitting is not primarily a data quantity problem but rather a limitation of the document representation itself, since averaging word vectors into a single document vector loses word order and local context information, which limits the classifier's ability to fully distinguish between the three classes. However compared to the TF-IDF model, the gap between training and validation F1 is noticeably smaller (0.78 vs 0.41 at 10%, compared to 0.98 vs 0.34 for TF-IDF), suggesting that Word2Vec generalizes better due to its ability to capture semantic relationships between words rather than relying on exact lexical matches.

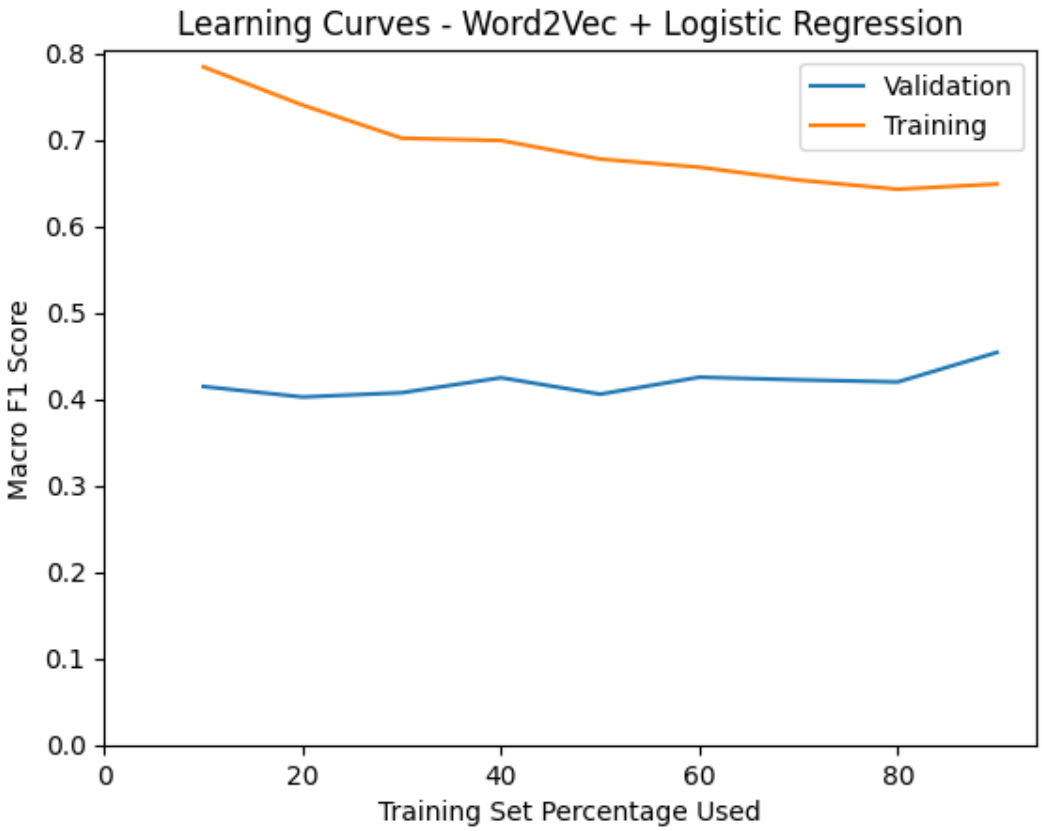


Figure 9: Learning Curves — Word2Vec + Logistic Regression (Best Model)

Training %	Train F1	Validation F1
10%	0.78	0.41
20%	0.74	0.40
30%	0.70	0.41
40%	0.70	0.42
50%	0.68	0.41
60%	0.67	0.43
70%	0.65	0.42
80%	0.64	0.42
90%	0.65	0.45

Table 7: Training and Validation Macro F1 Scores per Training Subset Size — Word2Vec + Logistic Regression (Best Model)

3.5.4. Confusion matrix.

A confusion matrix provides a detailed breakdown of model predictions per class, showing how many samples were correctly classified and how many were misclassified into other classes.

TF-IDF + Logistic Regression Part

As shown in Figure 10, the model performs well on the Ambivalent class, correctly classifying 161 out of 206 samples (78.2%), while misclassifying 39 as Clear Reply and only 6 as Clear Non-Reply. This is expected given that Ambivalent is the dominant class with 59.3% of the samples and the model naturally becomes more biased toward predicting it.

The model struggles significantly with the two minority classes. For Clear Non-Reply, only 6 out of 23 samples (26.1%) were correctly classified, with 14 misclassified as Ambivalent and 3 as Clear Reply. For Clear Reply, only 20 out of 79 samples (25.3%) were correctly classified, with 54 misclassified as Ambivalent and 5 as Clear Non-Reply.

The most common error pattern across both minority classes is misclassification toward Ambivalent, which is consistent with the class imbalance present in the dataset. These observations are directly consistent with the per-class precision, recall and F1-score reported in Section 3.5, where both Clear Non-Reply (F1=0.30) and Clear Reply (F1=0.28) scored significantly lower than Ambivalent (F1=0.74), confirming that the model heavily favors the majority class despite the use of class_weight='balanced'.

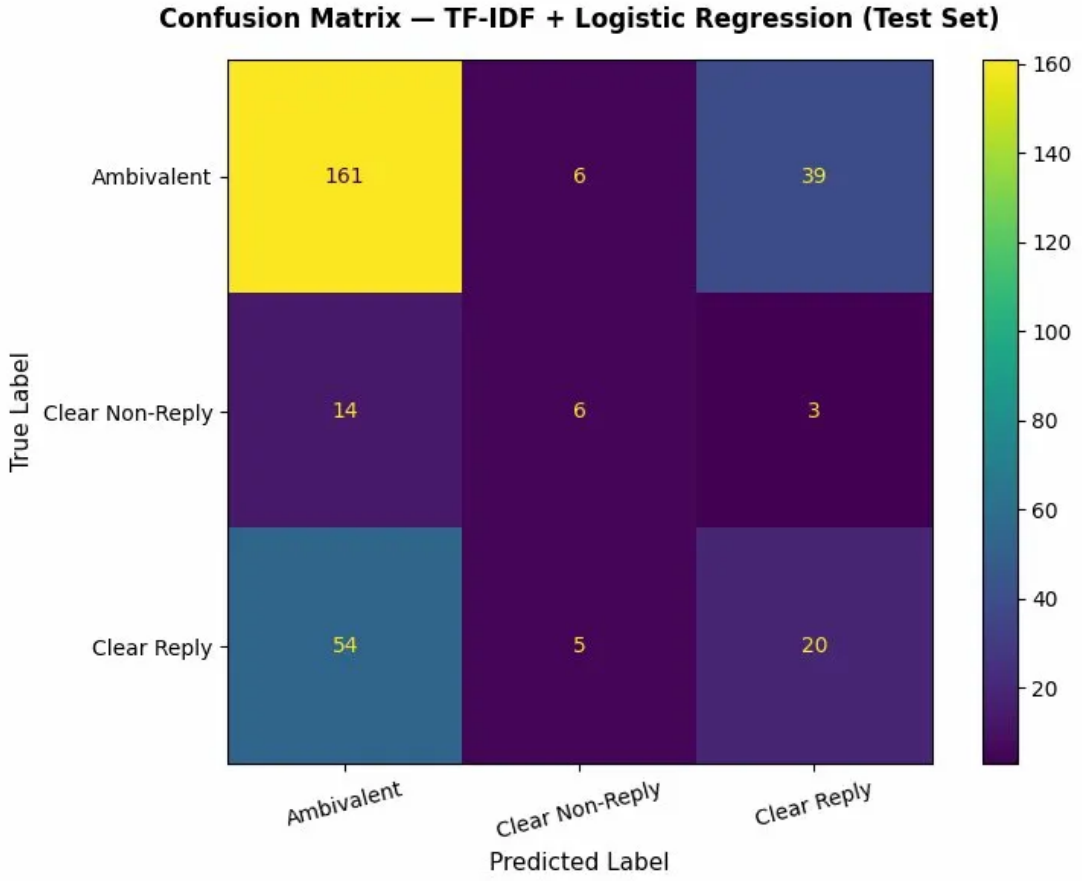


Figure 10: Confusion Matrix — TF-IDF + Logistic Regression (Test Set)

Word2Vec + Logistic Regression Part

As shown in Figure 11, the model performs well on the Ambivalent class, correctly classifying 131 out of 206 samples (63.6%), while misclassifying 61 as Clear Reply and

14 as Clear Non-Reply. This is expected given that Ambivalent is the dominant class with 59.3% of the samples and the model naturally becomes more biased toward predicting it.

The model struggles significantly with the two minority classes. For Clear Non-Reply, only 11 out of 23 samples (47.8%) were correctly classified, with 7 misclassified as Ambivalent and 5 as Clear Reply. For Clear Reply, only 28 out of 79 samples (35.4%) were correctly classified, with 39 misclassified as Ambivalent and 12 as Clear Non-Reply.

The most common error pattern across both minority classes is misclassification toward Ambivalent, which is consistent with the class imbalance present in the dataset. These observations are directly consistent with the per-class precision, recall and F1-score reported in Section 3.5, where both Clear Non-Reply (F1=0.40) and Clear Reply (F1=0.26) scored significantly lower than Ambivalent (F1=0.77), confirming that the model heavily favors the majority class despite the use of `class_weight='balanced'`.

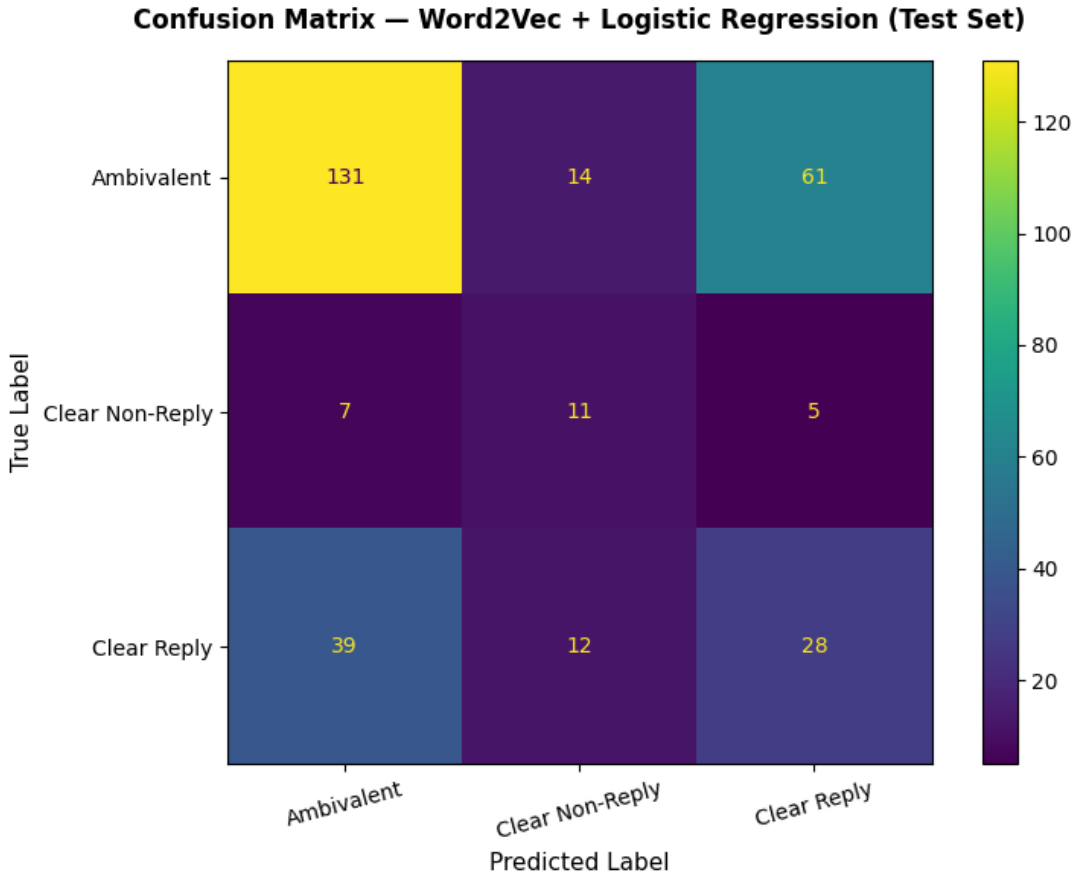


Figure 11: Confusion Matrix — Word2Vec + Logistic Regression (Test Set)

Comparison TF-IDF vs Word2Vec

The table below summarizes the per-class accuracy and Test Macro F1 for both models:

The comparison reveals an interesting tradeoff between the two models.

Class	**TF-IDF**	**Word2Vec**
Ambivalent	161/206 = 78.2%	131/206 = 63.6%
Clear Non-Reply	6/23 = 26.1%	11/23 = 47.8%
Clear Reply	20/79 = 25.3%	28/79 = 35.4%
Test Macro F1	0.4413	**0.4775**

Table 8: Per-class accuracy comparison: TF-IDF vs Word2Vec (Test Set)

TF-IDF performs better on the Ambivalent class (78.2% vs 63.6%), which is the majority class and therefore less critical for overall performance. Word2Vec on the other hand performs significantly better on both minority classes i.e. Clear Non-Reply (47.8% vs 26.1%) and Clear Reply (35.4% vs 25.3%). Since Macro F1 treats all three classes equally regardless of their size Word2Vec achieves a higher overall Test Macro F1 (0.4775 vs 0.4413), confirming it is the better model for this imbalanced classification task. This can be attributed to the fact that Word2Vec was trained from scratch on our own dataset, allowing it to learn word representations specifically tailored to the vocabulary and context of our data, rather than relying on pre-trained embeddings such as the Google News model.

4. Results and Overall Analysis

4.1. Results Analysis

Both models were evaluated on the local test set and on the Kaggle competition leaderboard. The TF-IDF model achieved a Test Macro F1 of 0.4413 on the local test set and to the Kaggle competition a score of 0.55 which meets the baseline score set, while the Word2Vec model achieved a Test Macro F1 of 0.4775 on the local test set and to the Kaggle competition a score of 0.58 which surpasses the baseline score set and is the best score I achieved across all experiments. This difference of scores in the local test set and the Kaggle competition is expected as the Kaggle leaderboard uses a different mechanism than the local Macro F1 test score.

Considering that the dataset is significantly imbalanced across the three clarity labels (Ambivalent being the dominant class (59.3%), followed by Clear Reply (30.4%) and Clear Non-Reply (10.3%)) and that the task involves political language, which is inherently linguistically complex, these scores represent a reasonable performance for the TF-IDF approach and a good performance for the Word2Vec approach.

Regarding the limitations of each model, the TF-IDF model relies on exact lexical matches and cannot capture deeper semantic relationships between words, which limits its ability to generalize to unseen data. The Word2Vec model while better at capturing semantic relationships, loses word order and local context information by averaging word vectors into a single document vector, which limits its ability to fully distinguish between the three classes.

Regarding hyperparameter tuning, all available parameters within both the TF-IDF and Word2Vec frameworks were exhaustively explored through GridSearchCV. Despite trying all possible combinations, no significantly better score was achieved for either model, suggesting that the parameter space has been fully explored and no fur-

ther hyperparameter optimization is possible.

This suggests that in order to achieve a better Macro F1 score improvements must come from the preprocessing stage rather than further hyperparameter tuning. One idea that could potentially boost performance for both models is POS-aware lemmatization, which reduces words to their base form using their grammatical role (e.g., "answered" → "answer", "running" → "run", "better" → "good"). POS-aware lemmatization uses the grammatical context of each word to produce linguistically correct base forms, which could reduce vocabulary size, limit vocabulary noise and help the model generalize better to unseen data. However due to time constraints this approach was not implemented.

4.1.1. Best trial.

TF-IDF + Logistic Regression Part

The best trial for the TF-IDF model was **GridSearch Experiment 2**, which used the "question: [question text] answer: [answer text]" concatenation format with the custom stopword removal mechanism applied. The best configuration found by GridSearchCV was C=0.9, solver=lbfgs, penalty=L2, class_weight=balanced and ngram_range=(1,2).

The following table summarizes the performance of the best trial across all three evaluation sets:

	Cross-Validation F1	**Validation F1**	**Test F1**
GridSearch Experiment 2	0.5555	0.5087	0.4413

Table 9: Performance Summary of the Best Trial (GridSearch Experiment 2)

The per-class results on the **validation set** are presented in Table 10:

	Precision	**Recall**	**F1-score**	**Support**
Ambivalent	0.67	0.68	0.68	301
Clear Non-Reply	0.43	0.38	0.41	52
Clear Reply	0.44	0.45	0.44	154
Accuracy			0.58	507
Macro avg	0.51	0.50	0.51	507

Table 10: Per-Class Results on the Validation Set Best Trial (GridSearch Experiment 2)

The per-class results on the **test set** are presented in Table 11:

	Precision	**Recall**	**F1-score**	**Support**
Ambivalent	0.70	0.78	0.74	206
Clear Non-Reply	0.35	0.26	0.30	23
Clear Reply	0.32	0.25	0.28	79
Accuracy			0.61	308
Macro avg	0.46	0.43	0.44	308

Table 11: Per-Class Results on the Test Set Best Trial (GridSearch Experiment 2)

As shown in both tables the model performs consistently better on the Ambivalent class across both validation and test sets, which is expected given its dominance in the dataset. The minority classes Clear Non-Reply and Clear Reply remain challenging, with notably lower F1-scores on the test set (0.30 and 0.28 respectively) compared to the validation set (0.41 and 0.44), further confirming the overfitting behaviour discussed in Section 3.3 and the misclassification patterns observed in the confusion matrix in Figure 10.

Word2Vec + Logistic Regression Part

The best trial for the Word2Vec model used the "question: [question text] answer: [answer text]" concatenation format with the custom stopword removal mechanism applied and vector_size=650, window=10, epochs=80 for the Word2Vec parameters. The best configuration found by GridSearchCV was C=0.7, solver=liblinear, penalty=l2, class_weight=balanced.

The following table summarizes the performance of the best trial across all three evaluation sets:

	Cross-Validation F1	**Validation F1**	**Test F1**
Word2Vec Best Trial	0.4829	0.4689	0.4775

Table 12: Performance Summary of the Best Trial (Word2Vec + Logistic Regression)

The per-class results on the **validation set** are presented in Table 13:

	Precision	**Recall**	**F1-score**	**Support**
Ambivalent	0.65	0.77	0.71	301
Clear Non-Reply	0.30	0.33	0.31	52
Clear Reply	0.51	0.31	0.39	154
Accuracy			0.59	507
Macro avg	0.49	0.47	0.47	507

Table 13: Per-Class Results on the Validation Set Best Trial (Word2Vec + Logistic Regression)

The per-class results on the **test set** are presented in Table 14:

	Precision	**Recall**	**F1-score**	**Support**
Ambivalent	0.71	0.84	0.77	206
Clear Non-Reply	0.41	0.39	0.40	23
Clear Reply	0.36	0.20	0.26	79
Accuracy			0.64	308
Macro avg	0.50	0.48	0.48	308

Table 14: Per-Class Results on the Test Set Best Trial (Word2Vec + Logistic Regression)

As shown in both tables, the model performs consistently better on the Ambivalent class across both validation and test sets, which is expected given its dominance in the dataset. The minority classes Clear Non-Reply and Clear Reply remain challenging,

with notably lower F1-scores on the test set (0.40 and 0.26 respectively) compared to the validation set (0.31 and 0.39), further confirming the overfitting behaviour discussed in Section 3.3 and the misclassification patterns observed in the confusion matrix in Figure 11.

5. Bibliography

References

[1] Plotly — Used for EDA and visual representation part
https://plotly.com/python/basic-charts/

[2] Data Mining Project 1 — Some EDA code was adapted from a previous project I did
https://github.com/EmilBa122/Data-Mining-Project-1

[3] Grid Search with Scikit-Learn Pipelines
https://apxml.com/courses/getting-started-with-scikit-learn/chapter-6-building-pipelines/grid-search-pipelines

[4] Inspired by the following: the `document_vector` function and the idea of training Word2Vec from scratch
https://www.kaggle.com/code/kstathou/word-embeddings-logistic-regression